



Phase 3

1. How JavaScript Works Under the Hood

Before anything else, burn these three truths into memory. Everything in this phase builds on them.

JavaScript is single-threaded

JS has **one** call stack and **one** thread of execution. It can do exactly **one thing at a time**. There is no "run two functions in parallel" inside the language itself.

JavaScript is synchronous (by default)

Code runs **top to bottom, line by line**. A line does not start until the previous one finishes.

JavaScript is interpreted *and* JIT-compiled

Old explanation: "JS is an interpreted language." Modern reality: engines use **JIT (Just-In-Time) compilation** — they interpret your code to start fast, then compile the frequently-used ("hot") parts into optimized machine code for speed. So it's a hybrid.

Common myth to kill: "JS is multi-threaded because async works." ❌

JS itself is single-threaded. Async behaviour (timers, network calls) is handled by the **environment** (browser or Node), *not* by the JS engine. We'll prove this in the next phase with the Event Loop.

2. JavaScript Engines

JS code can't run on its own — it needs an **engine**. An engine is a program (mostly written in C++) that reads your JavaScript and turns it into something the

machine understands.

Engine	Used in
V8	Chrome, Node.js, Edge (Google)
SpiderMonkey	Firefox (Mozilla)
JavaScriptCore	Safari (Apple)

Simple mental model: the engine is a translator that converts your English-like code into machine code.

What happens inside the engine (the pipeline)

When you run a JS file, the engine roughly does this:

1. **Tokenizing / Lexing** — breaks code into small pieces called *tokens*. `let x = 10;` → `let`, `x`, `=`, `10`, `;`
2. **Parsing → AST** — tokens are turned into a tree structure called the **Abstract Syntax Tree (AST)**.
3. **Compilation (JIT)** — in V8, the **Ignition** interpreter converts the AST into *bytecode* and runs it immediately. The **TurboFan** optimizer then recompiles hot code into fast machine code.
4. **Execution** — the code runs on the **Call Stack** (next sections).

💡 **Try it live in class:** open astexplorer.net and paste a small snippet to see the AST. Students find this genuinely eye-opening.

3. Execution Context

What is an Execution Context (EC)?

An **Execution Context** is the "environment" in which a piece of JavaScript code is evaluated and executed. Think of it as a sealed container that holds everything the code needs to run: its variables, functions, and the value of `this`.

Whenever JS runs code, it always runs *inside* an execution context.

Global Execution Context (GEC)

- Created **first**, automatically, the moment your program starts.
- There is only **one** GEC.
- In the browser, it creates the global `window` object, and `this` points to `window`.

Function Execution Context (FEC)

- A **new** FEC is created **every time a function is called**.
- Each function call gets its own private context with its own variables.
- When the function finishes, its FEC is destroyed (usually — closures are the exception, covered later 😊).

The Two Phases of Every Execution Context

This is one of the most important concepts in the whole phase. Every EC is built in **two phases**:

Phase 1 — Creation Phase (Memory allocation)

Before any line runs, JS scans the code and sets up memory:

- All `var` variables get memory and are set to `undefined`.
- All **function declarations** are stored fully in memory.
- The value of `this` is decided.

Phase 2 — Execution Phase (Code runs)

Now JS runs the code line by line, assigning real values to the variables it reserved in Phase 1.

```
var x = 10;
function greet() {
  console.log("Hello");
}
greet();
```

What happens:

- *Creation phase:* `x` → `undefined`, `greet` → full function stored, `this` → set.

- *Execution phase*: `x` becomes `10`, then `greet()` is called → a new FEC is created → "Hello" prints.

🔑 **The two-phase model is the real reason hoisting works.** Keep it in mind for Section 5.

4. The Call Stack

How JavaScript tracks function calls

The **Call Stack** is a data structure that tracks *which function is currently running*. It works on **LIFO** — **Last In, First Out**. The last function pushed in is the first one to pop out.

- When a function is **called** → it's **pushed** onto the stack.
- When a function **returns / finishes** → it's **popped** off.
- The bottom of the stack is always the Global Execution Context.

Visualizing the Call Stack

```
function one() {  
  two();  
  console.log("one done");  
}  
function two() {  
  three();  
  console.log("two done");  
}  
function three() {  
  console.log("three done");  
}  
one();
```

Step-by-step stack movement:

Step	Action	Stack (top → bottom)
1	<code>one()</code> called	one → GEC
2	<code>two()</code> called inside one	two → one → GEC
3	<code>three()</code> called inside two	three → two → one → GEC
4	three finishes → prints	(pops three) two → one → GEC
5	two finishes → prints	(pops two) one → GEC
6	one finishes → prints	(pops one) GEC

Output order:

```
three done
two done
one done
```

The functions *start* in the order `one, two, three`, but they *finish* in reverse — that's LIFO in action.

Stack Overflow — what causes it

If functions keep getting pushed without ever popping (usually **infinite recursion**), the stack runs out of space and the engine throws:

```
function loop() {
  loop(); // calls itself forever, never returns
}
loop(); // ❌ RangeError: Maximum call stack size exceeded
```

Every recursive function **must** have a base case that stops the recursion, otherwise → stack overflow.

5. Hoisting

Hoisting is JS's behaviour of *appearing* to move declarations to the top of their scope. **Important:** nothing physically moves — this is just the **Creation Phase** allocating memory before the code runs.

var hoisting (initialized as **undefined**)

var declarations are hoisted and set to **undefined**. So you can access them before the line they're written on — you just get **undefined**, not an error.

```
console.log(a); // undefined (not an error!)
var a = 5;
console.log(a); // 5
```

let and **const** hoisting — the Temporal Dead Zone (TDZ)

let and **const** are hoisted too — but they are **not** initialized. From the start of the scope until the line they're declared on, they sit in the **Temporal Dead Zone (TDZ)**. Accessing them there throws an error.

```
console.log(b); // ❌ ReferenceError: Cannot access 'b' before initialization
let b = 10;
```

Keyword	Hoisted?	Initial value before declaration	Accessing early gives
var	Yes	undefined	undefined
let	Yes	none (TDZ)	ReferenceError
const	Yes	none (TDZ)	ReferenceError

Function declarations vs function expressions

This difference catches a lot of people.

Function declaration → **fully hoisted**. You can call it before it's defined:

```
sayHi(); // ✅ "Hi" — works
function sayHi() {
  console.log("Hi");
}
```

Function expression → **follows the variable's hoisting rules**. The variable is hoisted, but the function is *not* assigned until that line runs:

```
sayBye(); // ❌ TypeError: sayBye is not a function
var sayBye = function () {
  console.log("Bye");
};
```

Here `sayBye` is hoisted as `undefined` (because of `var`), so calling `undefined()` throws a `TypeError`. With `let` / `const` it would throw a `ReferenceError` (TDZ) instead.

🔑 **Rule of thumb:** declarations are hoisted with their *body*; expressions are hoisted only as a *variable*.

6. Scope

Scope = the region of code where a variable is accessible.

Global scope

Declared outside any function/block. Accessible everywhere.

```
let appName = "MyApp"; // global
function show() {
  console.log(appName); // ✅ accessible
}
```

Function scope

Variables declared inside a function are only visible inside that function. `var`, `let`, and `const` are all function-scoped when declared inside a function.

```
function test() {
  var secret = 42;
}
console.log(secret); // ❌ ReferenceError – not visible outside
```

Block scope

A block is anything inside `{ }` (if, for, while, or a plain block). `let` and `const` are **block-scoped**. `var` is **NOT** — `var` ignores blocks.

```
{
  let a = 1;
  var b = 2;
}
console.log(b); // 2 (var leaked out of the block)
console.log(a); // ❌ ReferenceError (let stayed inside)
```

Lexical scope — what does "lexical" really mean?

Lexical means "based on where the code is *physically written*", not where or how it's called. A function's scope is decided by its **location in the source code** at the time you write it — not at runtime.

```
function outer() {
  let name = "Sara";
  function inner() {
    console.log(name); // inner can "see" outer's variables —
    // because of where it's written
  }
  inner();
}
outer(); // "Sara"
```

`inner` is written *inside* `outer`, so it has access to `outer`'s variables. This physical nesting is "lexical scoping".

7. The Scope Chain — how JS looks up variables

When you use a variable, JS looks for it in this order:

1. The **current** scope.
2. If not found, the **parent (outer)** scope.

3. Then the parent's parent... and so on, all the way up to the **global** scope.
4. If it's nowhere → `ReferenceError`.

This chain of "look outward until you find it" is the **Scope Chain**. It only goes **outward/upward**, never inward.

```
let a = 10;
function outer() {
  let b = 20;
  function inner() {
    let c = 30;
    console.log(a, b, c); // 10 20 30
  }
  inner();
}
outer();
```

- `inner` finds `c` in itself.
- `b` is found in its parent (`outer`).
- `a` is found in the global scope.

But the reverse is impossible — `outer` can never access `c`, because `c` lives in a deeper (inner) scope. The chain is **one-directional**.

8. Closures 🧠 (the star of this phase)

What is a closure? (the real definition)

A **closure** is a function bundled together with its surrounding **lexical environment**. In plain words: a function **remembers the variables from the scope where it was created**, even after that outer scope has finished executing.

Analogy: a closure is like a backpack. When a function is created, it packs a backpack of all the variables around it. Wherever that function travels later, it carries the backpack — so it can still use those variables.

Step-by-step trace of how a closure works

```

function counter() {
  let count = 0;           // (1) local variable
  return function () {     // (2) inner function is returned
    count++;
    return count;
  };
}

const inc = counter();     // (3) counter() runs and finishes
console.log(inc()); // 1    // (4)
console.log(inc()); // 2    // (5)
console.log(inc()); // 3    // (6)

```

What actually happens:

1. `counter()` is called → a new execution context is created with `count = 0`.
2. It returns the inner function. Normally, when `counter()` finishes, its context (and `count`) would be destroyed.
3. **But** the returned inner function still references `count`. So JS keeps `count` alive in memory — this preserved bundle is the closure.
4. Each call to `inc()` reaches into the *same* preserved `count` and increments it → 1, 2, 3.

The `count` variable is now **private** — nothing outside can touch it directly. That's the magic.

Practical use cases

1. Data privacy (private variables)

```

function createBankAccount() {
  let balance = 0; // private – cannot be accessed directly
  return {
    deposit(amount) { balance += amount; return balance; },
    getBalance() { return balance; }
  };
}

```

```

    };
  }
  const acc = createBankAccount();
  acc.deposit(100);
  console.log(acc.getBalance()); // 100
  console.log(acc.balance);      // undefined – truly private

```

2. Function factories

A function that builds and returns customized functions.

```

function multiplier(factor) {
  return function (number) {
    return number * factor;
  };
}

const double = multiplier(2);
const triple = multiplier(3);
console.log(double(5)); // 10
console.log(triple(5)); // 15

```

3. Counters — see the `counter()` example above. Each counter keeps its own independent count.

4. Currying — transforming a function with many arguments into a chain of functions each taking one argument.

```

function add(a) {
  return function (b) {
    return function (c) {
      return a + b + c;
    };
  };
}

console.log(add(1)(2)(3)); // 6

```

Common closure interview questions

Q1. What is a closure in one line?

A function that remembers and can access variables from its lexical scope even after the outer function has returned.

Q2. What will this print?

```
function outer() {  
  let x = 10;  
  return function () { return x; };  
}  
const fn = outer();  
console.log(fn()); // ?
```

Answer: `10`. Even though `outer()` has finished, the returned function closed over `x`.

Q3. Do two calls to the same factory share state?

```
const c1 = counter();  
const c2 = counter();  
console.log(c1()); // 1  
console.log(c1()); // 2  
console.log(c2()); // 1 – independent!
```

Answer: No. Each call to `counter()` creates a **fresh** closure with its own `count`.

The classic Closure + Loop problem (and why `let` solves it)

This is *the* most famous closure interview trap.

```
for (var i = 0; i < 3; i++) {  
  setTimeout(function () {  
    console.log(i);  
  }, 1000);  
}  
// Output: 3 3 3 🤯 (not 0 1 2)
```

Why? `var` is **function-scoped**, so there is only **one** shared `i`. By the time the `setTimeout` callbacks run (after the loop is over), `i` has already become `3`. All three closures point to the *same* `i`.

Fix 1 — use `let`:

```
for (let i = 0; i < 3; i++) {  
  setTimeout(function () {  
    console.log(i);  
  }, 1000);  
}  
// Output: 0 1 2 ✓
```

`let` is **block-scoped**, so each loop iteration creates a **new** `i`. Each closure captures its own copy. Problem solved.

Fix 2 — IIFE (the old-school way, before `let`):

```
for (var i = 0; i < 3; i++) {  
  (function (j) {  
    setTimeout(function () {  
      console.log(j);  
    }, 1000);  
  })(i);  
}  
// Output: 0 1 2 ✓
```

Each iteration calls an Immediately Invoked Function Expression that captures the current value of `i` as a fresh parameter `j`.

🔑 **Takeaway:** the loop problem is the ultimate proof that closures capture *variables*, not *values* — and that scope (`var` vs `let`) controls how many variables actually exist.

Quick Revision Table

Concept	One-line summary
Single-threaded	One call stack, one thing at a time
JIT-compiled	Interpreted first, then hot code compiled for speed
Execution Context	The environment where code runs (GEC + FEC)
Two phases	Creation (memory) → Execution (code)
Call Stack	LIFO structure tracking running functions
Stack overflow	Too many calls without returning (infinite recursion)
<code>var</code> hoisting	Hoisted as <code>undefined</code>
<code>let</code> / <code>const</code> hoisting	Hoisted but in TDZ → error if accessed early
Lexical scope	Scope decided by <i>where code is written</i>
Scope chain	Look outward/up until variable is found
Closure	Function + the variables it remembers from its birthplace

Practice Before Next Class

Predict the output of each, **then** run it to check:

```
// 1
console.log(x);
var x = 5;

// 2
foo();
function foo() { console.log("called"); }

// 3
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}

// 4
function make() {
  let secret = "hidden";
```

```
    return () => secret;
  }
  const reveal = make();
  console.log(reveal());
```

Recommended: Go back to your Phase 1 and Phase 2 examples and re-read them with this new lens. The "aha" moments — *"oh THAT'S why it worked!"* — are exactly what make this phase special. 💡

Happy learning! Master these and you'll debug faster, ace interviews, and write code you actually understand.